

Projet NoSQL MongoDB

Catalogue intelligent de produits e-commerce

1. Sujet choisi

Le sujet choisi est un catalogue intelligent de produits pour un site e-commerce. L'objectif est de créer une application capable de stocker des produits, des clients, des commandes, des catégories et des avis dans une base MongoDB, puis d'exposer des requêtes métiers à travers une API REST et une page web.

2. Objectifs du projet

- Modéliser une base NoSQL avec 5 collections.
- Insérer des données de test réalistes.
- Préparer 15 besoins utilisateurs.
- Créer 15 requêtes MongoDB utiles.
- Exposer les données via une API Express.
- Créer une interface web simple pour démontrer les requêtes.

3. Pourquoi MongoDB et NoSQL ?

MongoDB est adapté à ce projet car les produits d'un catalogue e-commerce ne possèdent pas tous les mêmes informations. Un ordinateur peut avoir une quantité de RAM, une capacité de stockage et une taille d'écran. Un casque peut avoir une autonomie, une connexion Bluetooth ou une réduction de bruit.

Avec MongoDB, les produits sont stockés sous forme de documents JSON. Le champ specs peut changer selon le type de produit. Ce modèle est plus flexible qu'un modèle SQL classique avec de nombreuses tables ou colonnes vides.

4. Collections utilisées

Collection	Rôle
categories	Stocke les catégories du catalogue.
products	Stocke les produits, prix, stocks, notes et caractéristiques.
customers	Stocke les clients, villes, fidélité et intérêts.
orders	Stocke les commandes avec les produits achetés.
reviews	Stocke les avis clients liés aux produits.

5. Modèle document / relations

categories -> products

customers -> orders

customers -> reviews

products -> reviews

orders -> items[] qui contiennent un snapshot des produits achetés

6. Exemple de document produit

```
{
  "name": "Casque AirSound Pro",
  "slug": "casque-airsound-pro",
  "brand": "Soundly",
  "categorySlug": "audio",
  "price": 149.99,
  "stock": 18,
  "sold": 72,
  "averageRating": 4.5,
  "tags": ["casque", "bluetooth", "audio"],
  "specs": {
    "battery": "35h",
    "noiseCancelling": true
  }
}
```

7. Besoins utilisateurs

1. Rechercher un produit par mot-clé.
2. Filtrer les produits par catégorie.
3. Voir uniquement les produits disponibles.
4. Trouver les produits les mieux notés.
5. Trouver des produits selon un budget.
6. Consulter la fiche détaillée d'un produit.
7. Lire les avis d'un produit.
8. Voir les produits en stock faible.
9. Voir les meilleures ventes.
10. Calculer le chiffre d'affaires total.
11. Analyser le chiffre d'affaires par catégorie.
12. Suivre les ventes par mois.
13. Voir les commandes à traiter.
14. Identifier les villes qui génèrent le plus de ventes.
15. Recommander des produits selon l'historique client.

8. Requêtes MongoDB

Les 15 requêtes MongoDB complètes sont fournies dans le fichier docs/requetes_mongodb.js.

Exemple 1 - Produits en stock faible

```
db.products.find(
  { stock: { $lte: 5 } },
  { name: 1, stock: 1, categorySlug: 1, price: 1 }
).sort({ stock: 1 });
```

Exemple 2 - Chiffre d'affaires par catégorie

```
db.orders.aggregate([
  { $match: { status: { $ne: 'cancelled' } } },
```

```

{ $unwind: '$items' },
{
  $group: {
    _id: '$items.categorySlug',
    revenue: { $sum: { $multiply: ['$items.price', '$items.quantity'] } },
    quantity: { $sum: '$items.quantity' }
  }
},
{ $sort: { revenue: -1 } }
});

```

9. API REST

Endpoint	Rôle
GET /api/dashboard	KPI, stock faible, produits bien notés.
GET /api/products	Recherche, filtres et tris du catalogue.
GET /api/products/:slug	Fiche complète d'un produit avec ses avis.
GET /api/recommendations/:customerId	Recommandations selon l'historique client.
GET /api/analytics?type=...	Analyses : top produits, CA par catégorie, ventes mensuelles, commandes à traiter.

10. Page web

Le front web se trouve dans le dossier public. Il permet de tester le tableau de bord, le catalogue filtré, les meilleures ventes, le chiffre d'affaires par catégorie, les commandes à traiter et les recommandations client.

11. Installation

```

npm install
copy .env.example .env
npm run seed
npm start

```

Front : <http://localhost:3000>

12. Conclusion

Ce projet respecte les attentes d'un cas d'utilisation NoSQL. Il contient une base MongoDB organisée en 5 collections, des données de test, 15 besoins utilisateurs, 15 requêtes MongoDB, une API REST et une page web de démonstration. Le choix de MongoDB est justifié par la flexibilité nécessaire pour gérer des produits aux caractéristiques différentes.